

RollbackLROnPlateau: An Experimental Investigation of Training Recovery

Experimental study of rollback-based learning-rate scheduling

Abstract

Plateau-based learning-rate schedulers reduce the learning rate when validation performance stops improving, but they normally continue from the model state at which the plateau was detected. This study investigates a simple modification: before reducing the learning rate, restore the model parameters to the best previously observed validation state. Two variants are evaluated on Fashion-MNIST: one resets AdamW’s optimizer state after rollback and the other retains the corresponding optimizer state. Each configuration is evaluated over 10 independent runs and compared with standard ReduceLROnPlateau. Under the selected experimental conditions, both rollback variants achieve lower mean best validation loss than the baseline, with mean losses of 0.2753 and 0.2743 versus 0.2846. The improvement is accompanied by increased training cost. Rollback events are also frequently followed by lower validation loss in the next epoch. A separate regression experiment provides a counterexample in which rollback and the standard scheduler perform similarly, suggesting that the observed benefit is task-dependent rather than a universal property of rollback scheduling.

1 Introduction

Learning-rate scheduling is a central component of neural-network optimization. A common strategy is to monitor a validation metric and reduce the learning rate when progress stalls. ReduceLROnPlateau implements this idea without changing the current model parameters. The present experiment asks whether a different response to a plateau can be useful: return to the best model state observed so far and only then reduce the learning rate.

The motivation is straightforward. During stochastic optimization, a model can reach a comparatively favorable validation state and subsequently move to a region with worse validation performance. If a plateau is detected after such movement, lowering the learning rate from the current state does not undo the movement. Rollback changes the starting point of the subsequent low-learning-rate phase.

The experiment is deliberately exploratory. Its objective is not to establish a new universally superior scheduler, but to determine whether this modification produces a measurable change in training behavior and to examine the role of optimizer state after restoration.

2 Research Question and Hypothesis

The primary research question is:

Does restoring the best previously observed model state before reducing the learning rate produce a more favorable optimization trajectory after a validation-loss plateau?

A secondary question concerns AdamW’s internal state:

After restoring model parameters, does resetting the optimizer state produce different behavior from retaining the optimizer state associated with the restored model state?

The hypothesis is that rollback may be beneficial when optimization has moved away from a previously favorable region. Since AdamW maintains internal moments in addition to model parameters, resetting or retaining those moments may also affect the subsequent trajectory.

3 Methodology

3.1 Dataset and preprocessing

The experiment uses Fashion-MNIST, a 10-class image-classification dataset with 60,000 training examples and 10,000 test examples. The 60,000 training examples are split into 80% training and 20% validation sets using a fixed split seed of 42. The 784 pixel features are scaled by dividing the original integer intensities by 255. The resulting tensors are represented as float32. The test set is not used for scheduler selection or the reported comparisons.

Mini-batches contain 250 examples. Training batches are shuffled, while validation batches are not.

3.2 Model

The fixed classifier is a fully connected network:

$$784 \rightarrow 512 \rightarrow 128 \rightarrow 64 \rightarrow 10.$$

Each hidden layer is followed by Batch Normalization, GELU activation, and Dropout. Dropout probabilities are 0.30, 0.25, and 0.20 for the three hidden layers, respectively. The output layer produces 10 class logits.

3.3 Optimization and training

All configurations use Cross-Entropy Loss and AdamW with an initial learning rate of 5×10^{-3} and weight decay of 10^{-3} . Training is allowed to continue for at most 100 epochs, with an early-stopping patience of 15 epochs. The scheduler patience is 3 epochs, the learning-rate reduction factor is 0.25, the improvement threshold is 10^{-4} , and the minimum learning rate is 10^{-6} .

Each configuration is trained independently for 10 runs. The recorded quantities are best validation loss, epoch of best validation loss, total epochs trained, training time, learning-rate changes, and rollback events.

3.4 Scheduling strategies

Three configurations are evaluated.

Table 1: Scheduler configurations.

Configuration	Patience	Optimizer state	Rollbacks
Rollback + Reset	3	Reset	4
Rollback + Retain	3	Retain	4
ReduceLROnPlateau	3	Standard	—

For the rollback scheduler, the best model state is stored whenever validation loss improves beyond the threshold. Once the patience criterion is reached, the stored model parameters are restored and the learning rate is reduced by the factor 0.25. With the Reset variant, the AdamW optimizer state is cleared. With the Retain variant, the optimizer state stored at the corresponding best model state is restored. A maximum of four rollback events is allowed per run. After this limit is reached, subsequent plateaus only reduce the learning rate.

4 Results

4.1 Overall validation performance

Table 2 summarizes the 10 independent runs for each configuration.

Table 2: Summary across 10 independent runs.

Configuration	Best loss	SD	Epochs	Time (s)
Rollback + Reset	0.2753	0.0042	45.7	136.2
Rollback + Retain	0.2743	0.0043	41.8	116.9
ReduceLROnPlateau	0.2846	0.0053	29.3	84.7

Both rollback configurations achieve lower mean best validation loss than the baseline. Relative to the mean loss of 0.2846 for ReduceLROnPlateau, the reductions are approximately 3.27% for Rollback + Reset and 3.62% for Rollback + Retain.

The per-run results in Figure 1 show that the difference is not produced by a single unusually good rollback run. The rollback configurations remain below the baseline across most independent runs, although run-to-run variation is visible.

Figure 2 presents the corresponding mean and standard deviation. The two rollback variants are close to one another, whereas the baseline has a higher mean loss and larger standard deviation.

4.2 Computational cost

The improved validation performance is accompanied by longer training. Rollback + Reset trains for 45.7 epochs on average, Rollback + Retain for 41.8 epochs, and ReduceLROnPlateau for 29.3 epochs. Mean wall-clock times are 136.2, 116.9, and 84.7 seconds, respectively.

The relationship between performance and training time is shown in Figure 3. The baseline occupies a lower-time region but generally achieves higher validation loss. Rollback + Retain tends to dominate Rollback + Reset within the observed rollback runs, requiring less time while obtaining a slightly lower mean loss.

Thus, the rollback result should not be summarized simply as “better.” Under these settings it is better in the measured validation-loss objective, but more expensive in training time.

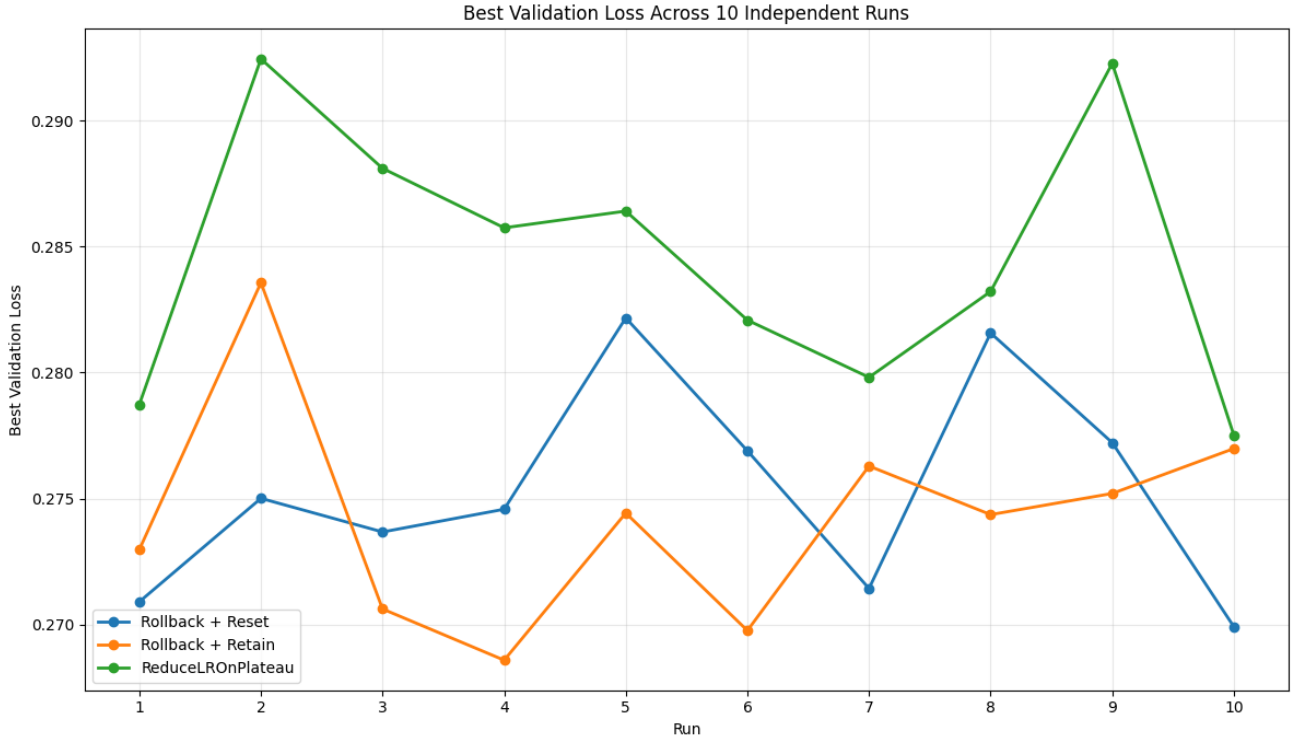


Figure 1: Best validation loss for each of the 10 independent runs. Lower values indicate better validation performance.

4.3 Learning-rate dynamics

The representative Run 1 trajectory in Figure 4 illustrates the difference in scheduling behavior. `ReduceLRonPlateau` begins reducing the learning rate earlier, while both rollback configurations remain at the initial rate for longer and subsequently continue through reductions over longer training trajectories.

The important distinction is therefore not merely the timing of the learning-rate reduction. A rollback event changes both the learning rate and the model state from which optimization resumes.

Across runs, the timing plot in Figure 5 similarly shows that the baseline generally responds to plateaus earlier, whereas rollback configurations produce learning-rate changes over a longer training interval.

4.4 Behavior immediately after rollback

There are 40 rollback events for each rollback configuration because the maximum of four rollbacks was reached in every run. For each event, the validation loss at the plateau-detection epoch is compared with the validation loss in the following epoch.

For Rollback + Reset, the mean next-epoch reduction is 0.01404, and 39 of 40 events (97.5%) are followed by a lower validation loss. For Rollback + Retain, the corresponding mean reduction is 0.01448, with 38 of 40 events (95.0%) followed by a lower validation loss.

This is evidence of a short-term association between rollback and subsequent improvement in this experiment. It is not evidence that every rollback leads to a new global best validation loss, nor does it establish causality independently of the learning-rate reduction that occurs at the same event.

4.5 Optimizer state: reset versus retain

Rollback + Retain achieves a mean best validation loss of 0.2743 compared with 0.2753 for Rollback + Reset. It also requires fewer epochs (41.8 versus 45.7) and less training time (116.9 versus 136.2 seconds).

The difference in validation loss is small relative to the run-to-run variability, and the two configurations overlap substantially. Therefore, these results provide only modest evidence that retaining the optimizer state is preferable under the tested conditions. They do not establish a general rule for AdamW state handling.

5 Discussion

The central observation is that restoring the best model state before reducing the learning rate changes the optimization trajectory in a measurable way. In this experiment, the change is associated with lower best validation loss, later best epochs,

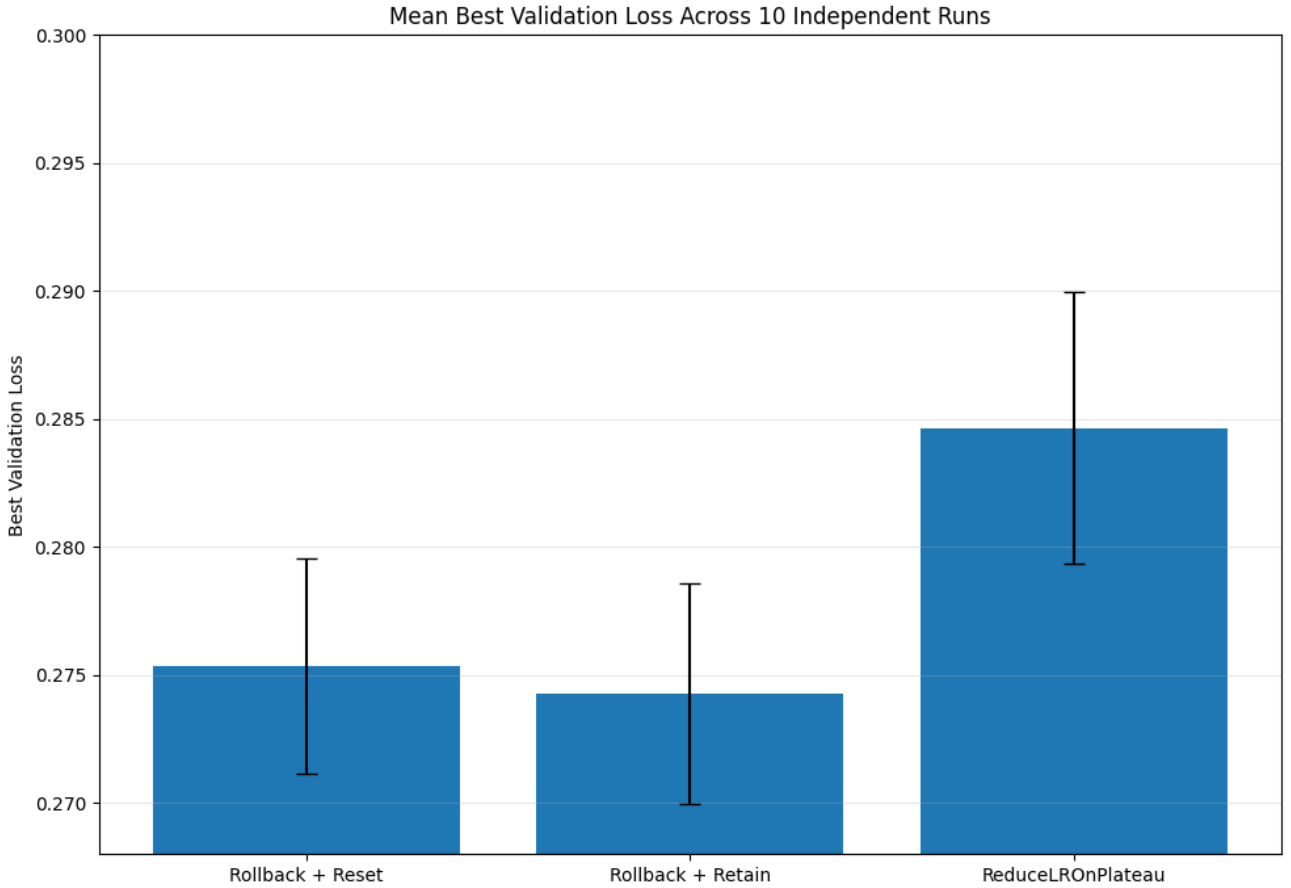


Figure 2: Mean best validation loss with standard deviation across the 10 independent runs.

and longer training.

One interpretation is that a plateau can occur after the model has moved away from a previously favorable validation state. Standard `ReduceLROnPlateau` decreases the step scale but leaves the parameters at their current location. Rollback instead returns the parameters to the best recorded state before reducing the step scale. If the current state is unfavorable, this can give the lower learning-rate phase a more favorable starting point.

The experiments do not directly establish this mechanism. In particular, the study does not measure distance in parameter space, curvature, gradient alignment, or whether the model has actually overshoot a favorable basin. The observed behavior is therefore consistent with, but does not prove, the proposed explanation.

The optimizer-state comparison also highlights an important distinction between parameter restoration and optimizer restoration. AdamW carries internal state beyond the model parameters. Restoring parameters without restoring or clearing this state creates a different optimization condition from restoring both. The modest difference between the two variants suggests that this distinction is worth investigating further, but the current sample is insufficient to make a general claim.

6 Counterexample: Heavy-Machinery Regression

A separate heavy-machinery regression experiment provides a useful counterexample. In that task, `RollbackLROnPlateau` and standard `ReduceLROnPlateau` achieved similar validation performance, approximately 0.47–0.48, and rollback events were not consistently followed by improved validation loss.

This observation is important because it prevents the Fashion-MNIST result from being interpreted as evidence that rollback is inherently superior. The benefit appears to depend on the training dynamics of the task, and the present experiment does not identify the precise conditions under which rollback is advantageous.

7 Limitations

The study has several limitations. First, each configuration uses only 10 independent runs, which is sufficient to expose run-to-run variation but is a small sample for broad statistical claims. Second, the main investigation uses one dataset and one architecture. Third, only one set of scheduler hyperparameters is evaluated, including a patience of 3, reduction factor of 0.25, minimum learning rate of 10^{-6} , and maximum of four rollbacks. Fourth, the runs are independent rather than paired

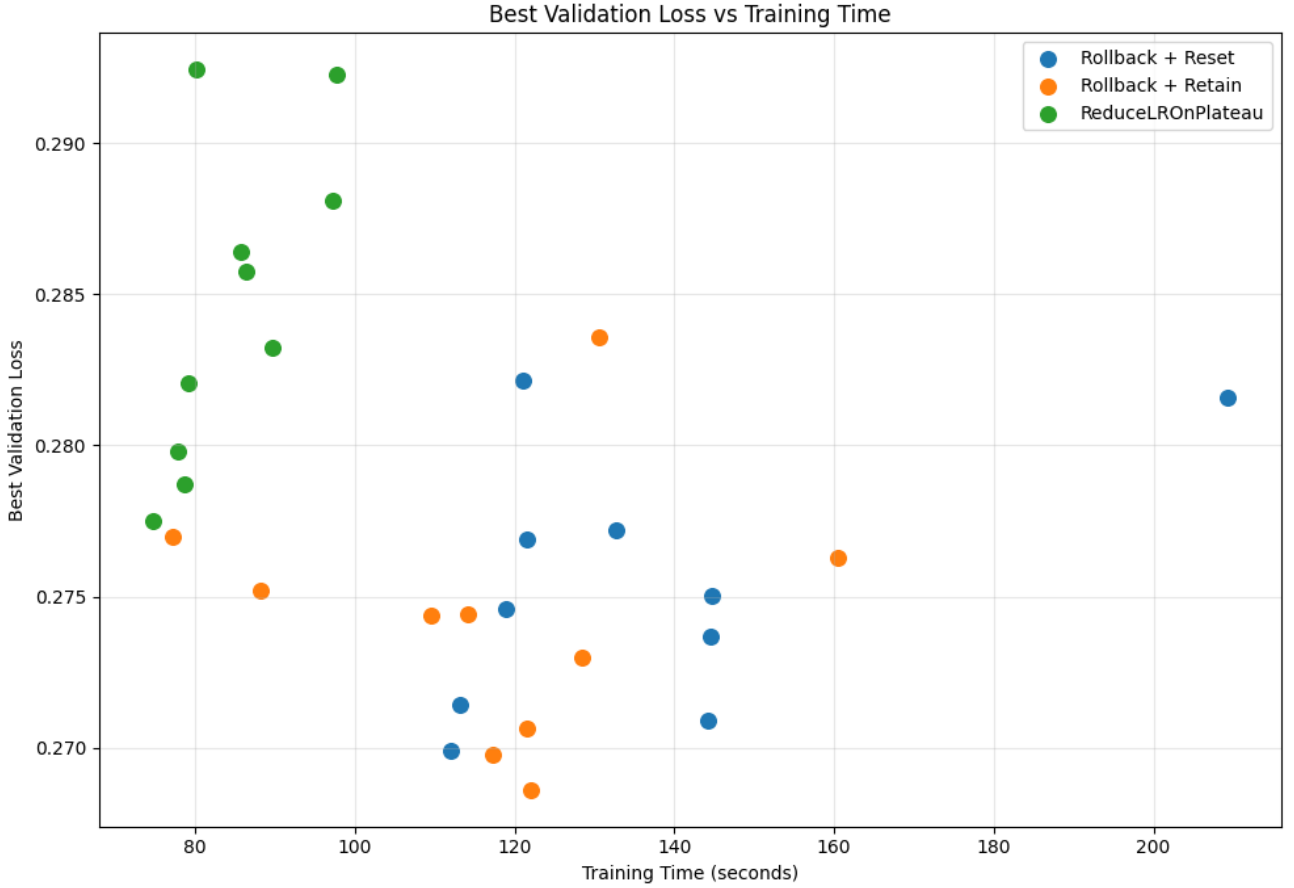


Figure 3: Best validation loss versus training time for individual runs. The experiment exhibits a performance–compute trade-off.

with identical random seeds across configurations. Fifth, rollback receives a larger effective computational budget because it generally trains for more epochs. Finally, the scheduler is a custom exploratory implementation, so the conclusions concern the tested implementation and experimental protocol rather than an established optimization method.

8 Conclusion

This study evaluated a rollback-based modification to plateau learning-rate scheduling in which the best previously observed model state is restored before reducing the learning rate. Across 10 independent Fashion-MNIST runs per configuration, both Rollback + Reset and Rollback + Retain achieved lower mean best validation loss than standard ReduceLROnPlateau. Rollback + Retain obtained the lowest mean loss at 0.2743, compared with 0.2753 for Rollback + Reset and 0.2846 for the baseline.

The improvement was accompanied by higher computational cost. Rollback + Retain and Rollback + Reset trained for 41.8 and 45.7 epochs on average, compared with 29.3 for the baseline. Rollback events were also frequently followed by lower validation loss in the next epoch, although this short-term behavior does not imply that every rollback produces the eventual best state.

The separate regression counterexample indicates that the observed advantage is not universal. The most defensible conclusion is therefore that rollback can materially alter plateau recovery dynamics and can be beneficial under some optimization conditions, but its usefulness is task-dependent.

The experiment consequently motivates further controlled studies of the relationship between rollback, optimizer state, learning-rate reduction, and the geometry of the optimization trajectory.

References

- [1] H. Xiao, K. Rasul, and R. Vollgraf, “Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms,” *arXiv preprint arXiv:1708.07747*, 2017.
- [2] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” *International Conference on Learning Representations (ICLR)*, 2019.

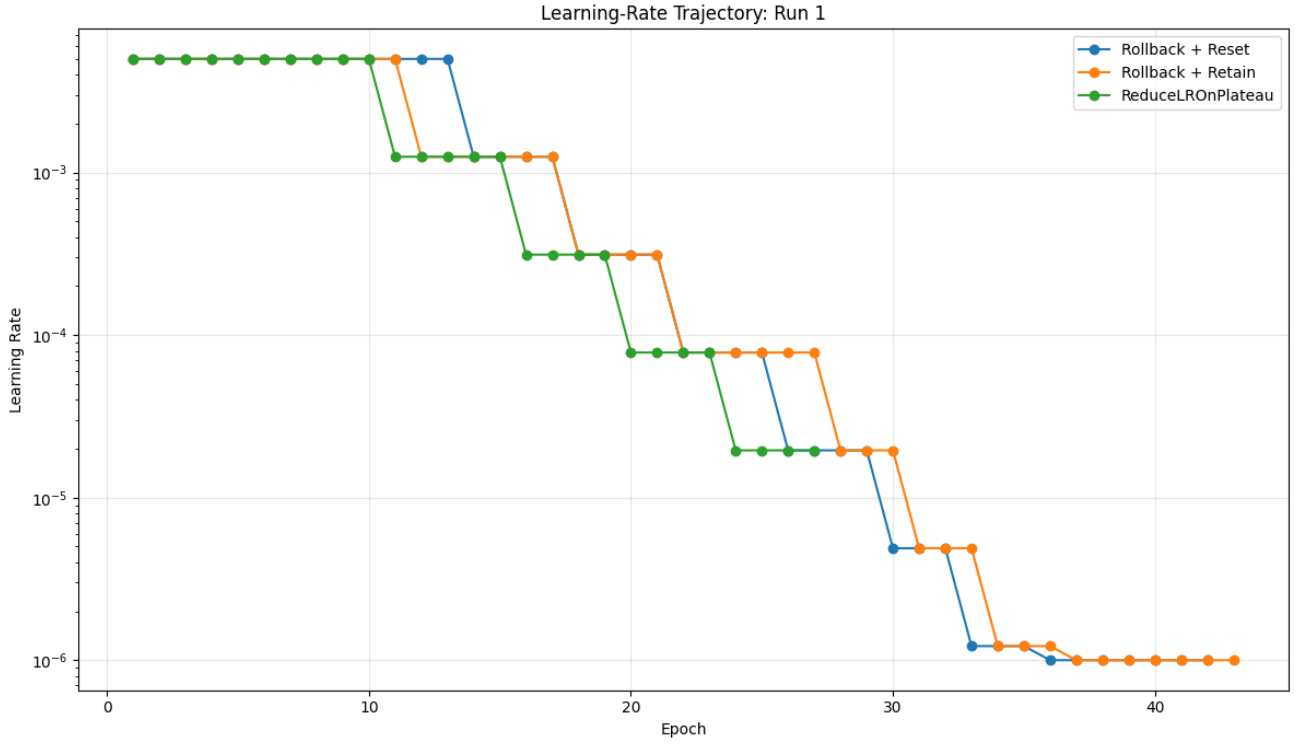


Figure 4: Learning-rate trajectory for Run 1. The logarithmic vertical axis shows the successive learning-rate reductions.

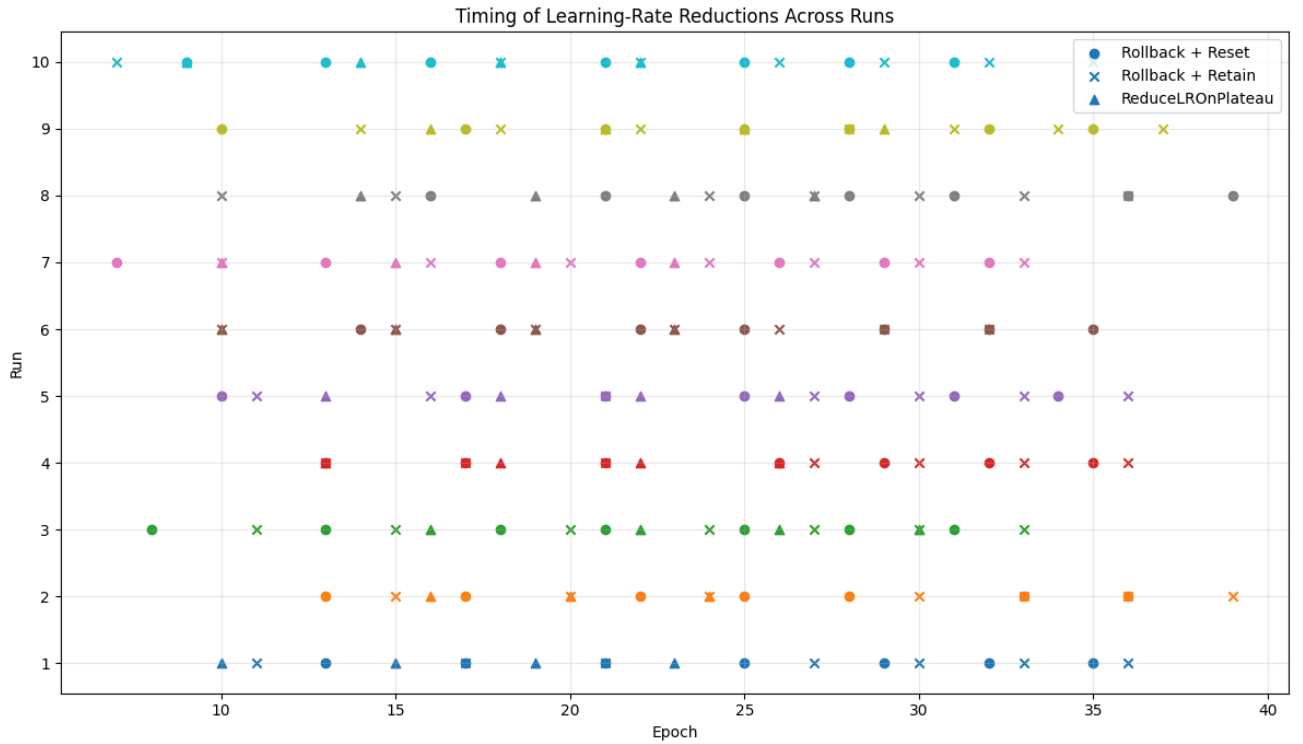


Figure 5: Epochs at which learning-rate reductions occur across the 10 runs. Different markers distinguish the three configurations.